

Collabnix

Helm Cheatsheet [2024 Updated]

While the [Helm docs](#) has extensive information on every command there is with Helm, there are a few commands that you will use regularly enough that it is important to have them memorized. This will also come in handy if you were ever to face an interview where you get asked questions about various Helm commands. It is unlikely that the interviewers will ask you to write multi-line complex commands since you absolutely will have to look through the documentation before you can get there. Instead, the commands they will expect you to know will be the basics that go on to show that you know and have used Helm in the past. First off, you should be familiar with Helm. If you aren't make sure to take a look at [Helm101](#) for an in-depth tutorial.

Before we begin, if you have consulted the [Kubernetes cheat sheet](#) before, you will notice that there are a large number of similarities between the flags, keywords, and commands. So let's start off on familiar ground by looking at these.

Helm flags

```
-A
```

`-A` stands for "all namespaces" (`--all-namespaces`) and is used to look for resources across all namespaces. So, commands such as

```
helm list -A
```

will list all the charts applying across all namespaces. Another flag which shows everything with no filter applied is `-a`, which can be used in the same way.

```
-f
```

This is used to specify a file. A command such as

```
helm install -f myvals.yaml ./mychart
```

will install the chart `mychart` and override the chart values with the values from the file `myvals.yaml`. However, this same flag can be used in a different context to apply a filter. An example of this would be if you were running the `list command`:

```
helm list -f "gitlab"
```

This would list all the charts with the word `gitlab` in their name.

```
-n
```

This is used to specify the namespace, and is short for `--namespace`.

```
-o
```

shorthand for `--output`, which formats the output displayed. So if you wanted the output to be in JSON format, use

```
helm ... -o json
```

The flags that were discussed up to now are ones that match flags on Kubernetes. We will now move to flags that are specific to Helm.

```
-g
```

When you are installing a Helm chart, the chart would be installed as a release. This means that you can install the same chart multiple times with different names on the same host. To facilitate this, you need to assign a name to the helm chart. However, you can use the above flag to automatically generate a name for you.

```
helm install examples/hello-world -g
```

will install a chart with a generated name that includes the chart name as a prefix. Also, note that `-g` is the shortened form of `--generate-name`.

```
--password  
--username
```

These two flags are useful if you are connecting to a private repository that requires authorization. Example:

```
helm install privatereponame/chartname --username user --password  
password --generate-name
```

If you are logging in to a private Helm registry with the `helm registry login` command, you can use the shorthand `-p` and `-u` instead.

```
--version
```

This flag is a common one that you may have used with other CLI tools to get the version of the tool. However, the flag above cannot be used this way. It is to be used to specify a version of a chart:

```
helm upgrade release-1 jenkins --version [version-number]
```

The above command will install the version of the Jenkins chart specified.

```
-h
```

Finally, we have `-h`, which allows you to get help on a number of helm commands. So using

```
helm get all -h
```

Will show you information on `get all`:

This command prints a human-readable collection of information about the notes, hooks, supplied values, and generated manifest file of the given release.

Usage:

helm get all RELEASE_NAME [flags]

There is a large number of other flags, but the ones mentioned above are the ones that are mostly used. You will be expected to know at least some of these flags while the rest can be easily accessed via the documentation.

Accessing the documentation

Before we continue, it is important to understand how you can navigate the official documentation since you will have to refer to it sooner or later. The site to the main doc page is <https://helm.sh/docs/>. You have several guides here along with a sidebar

on the left that has a section `helm commands`. This section expands to show several commands that consist of 90% of the commands you will be using. Clicking on each command leads to the page that describes the command. At the bottom of each page, you will find the full list of flags that can be used with the command along with the flags it inherited from its parent (which are also usable). You will find all of the flags mentioned above (and many more) in this section. At the very bottom, you will find commands that are similar to the command you are referring to.

So if you want to do something with a command, but aren't clear on the options, the helm docs are where you need to go.

Helm commands

Now that we have gone through flags which can apply to multiple commands, let's take a look at the various commands out there.

Before you do anything, you need to install a chart, and before you do that, you need to add the chart repository. So the first command you need to know is:

```
helm repo add
```

Example:

```
helm repo add jenkins https://charts.jenkins.io  
helm repo update
```

Note that no one will expect you to memorize the names of any repositories. Instead, if you are ever told to add a repository, go to artifacthub.io and search for the package. Most packages come with a readme that will tell you exactly how to get it up and running ([example](#)). Another alternative is:

```
helm search
```

This allows you to search for charts from the command line. There are a few variations such as:

```
helm search hub
```

Allows you to search artifact hub from the command line.

```
helm search hub jenkins
```

will give you all the charts on artifact hub with Jenkins in its name.

```
helm search repo
```

Allows you to search all installed repos in your local system for the chart. Note that you need to have the chart already added with `helm repo add` for anything to show up with this command.

Once you have the chart, you need to install it:

```
helm install
```

The most basic form of this command is:

```
helm install jenkins jenkins/jenkins
```

The above command installs the Jenkins chart and sets up all resources needed to run a Jenkins instance on your Kubernetes cluster. However, by default, the Jenkins service is of type `ClusterIP`, meaning that nothing outside of the clusters' localhost will be able to access it. To change this, you need to override the values in a Helm chart. This will be the case for most charts out there, where you will have to override values to make the chart cater to your specific situation. The most common way to do this is to override the chart values with separate values.yaml file:

```
helm install -f myvalues.yaml jenkins jenkins/jenkins
```

Since this format is so commonly used, it makes sense to memorize this command. There are also other formats:

```
helm install --set name=prod myredis ./redis
```

The above sets the values in line. The complete list of ways to use `helm install` can be found in the [docs](#). Once your chart is installed, you can go ahead and list them:

```
helm list
```

The command can be used in combination with the flags discussed above.

The next command to look at is:

```
helm show
```

Helm show is used to describe specific resources, and if you were to try and use this command with no prior experience, you will likely run into all sorts of errors, even if you are reading the documentation. This is because the arguments you have to

provide to these commands are not in the same format as the arguments provided to other Helm commands. If you want information about a chart, you need to use:

```
helm show chart <repo-name>/<chart-name>
```

The chart name alone is not enough. `helm show chart gitlab-runner` would be wrong, it should be `helm show chart gitlab/gitlab-runner`.

You can also use this command to show the values of a chart:

```
helm show values
```

You can use this command with a number of the flags that were previously discussed. The result of this command would be yaml formatted output that shows the values derived from the chart. This is useful if you want to quickly reference the values for a particular chart. However, the GitHub repos of many available charts also provide the `values.yaml`, which is easier to access and is much better commented. For example, if you wanted to see the overridable values for Jenkins, you could go to [the related file](#) in GitHub.

Another important command is:

```
helm status
```

This command allows you to see the detailed status of an installed chart release. For example:

```
helm status gitlab-runner
```

Will give you an output such as:


```
NAME: gitlab-runner
LAST DEPLOYED: Thu Feb 16 14:48:33 2023
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
Your GitLab Runner should now be registered against the GitLab
instance reachable at: "http://localhost:4500/"
```

The notes section is particularly useful since a lot of chart authors mention any specific details of the chart here.

Another command that is good to know is:

```
helm history <release-name>
```

This will give you the release history.

Now that the chart is installed, let's talk about how we can update it.

```
helm upgrade
```

is the command you should use in this situation, and it releases a new version of the chart. Note that you should specify the name of the release since it is the release that will be changing. You also need to specify the chart name:

```
helm upgrade -f values.yaml jenkins ./jenkins
```

You can also use `-f` to override values by passing in a yaml (or set overriding values in-line). You can also use most of the flags used by `helm install` here as well. You could also use the flag `--install` at the end of the command if you want to upgrade if the release exists or install if it doesn't.

Next, we get to chart creation. Note that creating a chart is somewhat complicated, and it is unlikely you will be asked to create charts in a situation such as an interview. On occasions when you have to create a chart, you should always take your time to read through the doc so you don't make any mistakes. However, it is important to keep the most basic create command in memory:

```
helm create <chart_name>
```

This will create a directory structure with the chart name containing all the boilerplate configuration files that you will need to create a complete chart.

Now that we have gone through the installation commands, let's look at the uninstallation commands. You can uninstall a release the same way you installed it:

```
helm uninstall <release>
```

You can remove a repo the same way you added it:

```
helm repo remove <repo-name>
```

Conclusion

This covers a large number of Helm commands that you get to use on a daily basis. We outlined the important flags that are worth remembering since they can be used in combination with a number of commands, as well as common commands that you can easily remember so that you don't have to consult the documentation during

your day-to-day work with Helm. Of course, if there is something that you are unclear about, the docs are your best option, and we covered the most optimal way to browse the docs. We also took a look at how to slightly more advanced commands that you will use sooner or later, but not on a daily basis.

So if you ever need a quick refresher on Helm commands, you can come back and consult this cheat sheet. If you need a full course on Helm, there is the [Helm section](#).

[Join our Discord Server](#)